

FinBridge 개발 가이드

FinBridge

2026-04-26

CONTENTS

FinBridge 개발문서	
1. 시스템 개요	
2. 패키지 구조	
주요 책임	
3. 요청 처리 흐름	
3.1 POST /api/integrate	
3.2 IntegrationService.processIntegration()	
3.3 GET /api/integrate/{requestId}	
3.4 GET /api/logs	
3.5 POST /api/integrate/{requestId}/retry	
3.6 인터페이스 등록/설정 API	
3.7 모니터링/성능관리 API	
4. 병렬 처리 구조	
4.1 CompletableFuture	
4.2 integrationTaskExecutor	
4.3 35초 timeout	
5. 트랜잭션 설계	
5.1 원칙	
5.2 실제 구현	

6. 결과 저장 책임	
7. 어댑터 설계	
7.1 SOAP	
7.2 Kafka	
7.3 SFTP	
7.4 Batch	
7.5 REST	
8. Kafka 설계	
8.1 topic	
8.2 docker-compose listener	
9. SFTP 설계	
10. Batch 설계	
11. DB와 Flyway	
11.1 도메인 테이블	
11.2 인덱스	
11.3 JPA 설정	
12. API 명세	
12.1 POST /api/integrate	
12.2 GET /api/integrate/{requestId}	
12.3 GET /api/logs	
12.4 POST /api/integrate/{requestId}/retry	

12.5 /api/interfaces	
12.6 모니터링과 성능관리	
13. 테스트 전략	
13.1 Unit Test	
13.2 H2 기반 Spring Integration Test	
13.3 Docker E2E Test	
14. 주요 장애 대응 포인트	
SFTP HostKey has been changed	
SFTP Permission denied	
Kafka host app connection 실패	
Batch metadata table 오류	
/api/logs 500 오류	
15. 운영 전 개선 포인트	
16. 유지보수 체크리스트	

FinBridge 개발문서

문서 목적: FinBridge 코드를 이해하고 유지보수하기 위한 내부 개발 문서

작성 기준: 현재 브랜치의 최신 구현

대상 독자: 백엔드 개발자, 리뷰어, 포트폴리오 평가자

1. 시스템 개요

FinBridge는 금융 IT 인터페이스 통합관리 시스템이다.

클라이언트가 `POST /api/integrate` 로 통합 요청을 보내면, 서버는 요청된 프로토콜을 확인한 뒤 등록된 인터페이스 설정을 조회한다. `enabled=false` 인 프로토콜은 어댑터를 실행하지 않고 `DISABLED` 결과로 확정한다. 활성화된 프로토콜은 SOAP, Kafka, SFTP, Batch, REST 어댑터를 병렬로 실행한다. 각 어댑터는 실행 결과만 DTO로 반환하고, 최종 `ProtocolResult` 저장과 `SystemLog` 저장은 `IntegrationService` 가 중앙에서 처리한다.

핵심 흐름은 다음과 같다.

```
Client
|
v
IntegrationController
|
| 1. protocol validation
v
IntegrationService
|
| 2. requestId 생성, IntegrationRequest 저장, INITIATED 로그 저장
|
| 3. InterfaceConfig 조회(enabled, endpoint, timeoutMs)
|
| 4. CompletableFuture + integrationTaskExecutor로 활성 어댑터 병렬 실행
|
+--> SoapAdapterService -> LegacySoapService
+--> KafkaAdapterService -> Kafka topic publish
+--> SftpAdapterService -> SFTP upload
+--> BatchAdapterService -> Spring Batch JobLauncher
+--> RestAdapterService -> LegacyRestService
|
| 5. 35초 timeout 기준으로 결과 수집
|
| 6. ProtocolResult 저장, SystemLog 저장, overallStatus 계산
v
IntegrationResponseDTO
```

2. 패키지 구조

```

src/main/java/com/finbridge
├── FinbridgeApplication.java
├── config
│   ├── AppConfig.java
│   ├── BatchJobConfig.java
│   ├── IntegrationExecutorConfig.java
│   ├── KafkaConfig.java
│   ├── SoapConfig.java
│   └── WebConfig.java
├── controller
│   ├── IntegrationController.java
│   ├── InterfaceConfigController.java
│   └── MonitoringController.java
├── model
│   ├── dto
│   │   ├── AdapterExecutionConfig.java
│   │   ├── InterfaceConfigDTO.java
│   │   ├── IntegrationRequestDTO.java
│   │   ├── IntegrationResponseDTO.java
│   │   ├── LogResponseDTO.java
│   │   ├── MonitoringSummaryDTO.java
│   │   ├── ProtocolPerformanceDTO.java
│   │   ├── ProtocolResultDTO.java
│   │   ├── RetryRequestDTO.java
│   │   └── RetryResponseDTO.java
│   ├── entity
│   │   ├── InterfaceConfig.java
│   │   ├── IntegrationRequest.java
│   │   ├── ProtocolResult.java
│   │   └── SystemLog.java
│   └── enums
│       ├── EventType.java
│       ├── OverallStatus.java
│       ├── ProtocolType.java
│       ├── RequestStatus.java
│       └── ResultStatus.java
├── repository
│   ├── InterfaceConfigRepository.java
│   ├── IntegrationRequestRepository.java
│   ├── ProtocolResultRepository.java
│   └── SystemLogRepository.java
├── service
│   ├── InterfaceConfigService.java
│   ├── IntegrationService.java
│   ├── KafkaConsumerService.java
│   ├── MonitoringService.java
│   └── adapter
│       └── BatchAdapterService.java

```

```

├── KafkaAdapterService.java
├── ProtocolAdapter.java
├── RestAdapterService.java
├── SftpAdapterService.java
├── SoapAdapterService.java
├── legacy
│   ├── LegacyRestService.java
│   └── LegacySoapService.java
├── soap
│   ├── IntegrationSoapRequest.java
│   ├── IntegrationSoapResponse.java
│   ├── SoapEndpoint.java
│   └── package-info.java

```

주요 책임

영역	책임
controller	API 요청 검증, 응답 상태 코드 결정
service.IntegrationService	통합 요청 생명주기, 병렬 실행, 결과 저장, 로그 저장
service.InterfaceConfigService	인터페이스 등록/수정/조회와 설정 검증
service.MonitoringService	요청 현황과 프로토콜별 성능 통계 조회
service.adapter	프로토콜별 실행 로직
service.legacy	로컬 데모용 REST/SOAP 레거시 처리
config	Kafka, Batch, SOAP, CORS, Executor 설정
repository	JPA 기반 DB 접근
model.entity	DB 테이블 매핑
model.dto	API 요청/응답 모델

3. 요청 처리 흐름

3.1 POST /API/INTEGRATE

진입점은 `IntegrationController.integrate()` 이다.

처리 단계:

1. `protocols`가 null 또는 empty인지 검사한다.
2. 각 프로토콜 문자열을 trim 후 uppercase로 정규화한다.
3. null, blank, 미지원 프로토콜이 있으면 400 Bad Request를 반환한다.
4. 정상 요청이면 `IntegrationService.processIntegration()`을 호출한다.

지원 프로토콜:

SOAP, KAFKA, SFTP, BATCH, REST

소문자 요청은 정상화된다.

```
{
  "protocols": ["rest", "sftp"],
  "payload": {
    "customerId": "C-1001"
  }
}
```

위 요청은 내부적으로 `REST`, `SFTP`로 변환된다.

3.2 INTEGRATIONSERVICE.PROCESSINTEGRATION()

`processIntegration()`은 전체 오케스트레이션을 담당한다.

중요한 점은 이 메서드 전체에 `@Transactional`을 사용하지 않는다는 것이다. 외부 시스템 호출이 포함된 병렬 실행 구간에서 DB 트랜잭션과 커넥션을 오래 잡지 않기 위해서다.

처리 순서:

1. UUID 기반 `requestId` 생성
2. `createInitialRequest()` 호출
 - `integration_requests` 저장
 - `INITIATED` 시스템 로그 저장
 - 요청 상태를 `PROCESSING`으로 변경
3. 요청된 프로토콜별 `InterfaceConfig` 조회
 - 설정 row가 없으면 기존 기본 동작으로 실행
 - 설정이 있고 `enabled=true`인 row가 있으면 해당 설정을 `Adapter`에 전달
 - 설정이 있지만 모두 `enabled=false`이면 `Adapter`를 실행하지 않고 `FAILED / DISABLED` 결과 생성

4. 활성 프로토콜별 `CompletableFuture` 생성
5. `integrationTaskExecutor` 로 어댑터 병렬 실행
6. `CompletableFuture.allOf(...).orTimeout(35, TimeUnit.SECONDS)` 로 최대 35초 대기
7. 완료된 결과를 `ProtocolResultDTO` 로 수집
8. timeout 또는 예외는 실패 결과로 변환
9. `determineOverallStatus()` 로 전체 상태 계산
10. `saveFinalResults()` 호출
 - 프로토콜별 `ProtocolResult` 저장
 - 프로토콜별 `SystemLog` 저장
 - `integration_requests` 를 `COMPLETED` 로 변경
 - 최종 완료 로그 저장
11. `IntegrationResponseDTO` 반환

3.3 GET /API/INTEGRATE/{REQUESTID}

`IntegrationController.getStatus()` 가 `IntegrationService.getStatus()` 를 호출한다.

조회 흐름:

1. `IntegrationRequestRepository.findById(requestId)` 로 요청 조회
2. 없으면 404 Not Found
3. 있으면 `ProtocolResultRepository.findById(requestId)` 로 프로토콜 결과 조회
4. `IntegrationResponseDTO` 로 반환

3.4 GET /API/LOGS

`IntegrationController.getLogs()` 에서 limit, offset, protocol query parameter를 검증한다.

검증 규칙:

- `limit <= 0` 이면 400
- `offset < 0` 이면 400
- 잘못된 `protocol` 이면 400
- `protocol` 은 소문자 입력도 uppercase 정규화 후 처리

서비스는 `OffsetBasedPageRequest` 로 offset 기반 페이지네이션을 구성한다.

3.5 POST /API/INTEGRATE/{REQUESTID}/RETRY

`IntegrationController.retry()` 가 `IntegrationService.retryIntegration()` 을 호출한다.

처리 흐름:

1. 원본 `requestId` 로 `IntegrationRequest` 를 조회한다.
2. 원본 요청이 없으면 404를 반환한다.
3. 요청 body의 `protocols` 가 있으면 해당 프로토콜만 재처리 대상으로 사용한다.
4. `protocols` 가 없거나 비어 있으면 원본 요청의 `ProtocolResult` 중 `SUCCESS` 가 아닌 프로토콜을 자동 선택한다.
5. 재처리 대상이 없으면 400을 반환한다.
6. 원본 payload를 복원해 새 `IntegrationRequestDTO` 를 만든다.
7. `processIntegration()` 을 다시 호출해 새 `requestId` 로 독립 실행한다.

재처리는 기존 `ProtocolResult` 를 덮어쓰지 않는다. 새 요청으로 저장되며 응답은 `originalRequestId`, `retryRequestId`, `retryResponse` 를 포함한다.

3.6 인터페이스 등록/설정 API

`InterfaceConfigController` 는 `/api/interfaces` 하위 API를 제공한다.

API	설명
<code>GET /api/interfaces</code>	전체 인터페이스 설정 조회
<code>GET /api/interfaces?protocol=SFTP</code>	특정 프로토콜 설정 조회
<code>POST /api/interfaces</code>	인터페이스 설정 생성
<code>PUT /api/interfaces/{id}</code>	인터페이스 설정 수정

검증 규칙:

- `protocol` 은 필수이며 `SOAP`, `KAFKA`, `SFTP`, `BATCH`, `REST` 중 하나여야 한다.
- `interfaceName` 은 필수다.
- `endpoint` 는 필수다.
- `timeoutMs` 는 null이거나 1 이상이어야 한다.
- `enabled` 가 null이면 `true`로 저장한다.

현재 실행 흐름에서 프로토콜별 설정 row가 여러 개일 수 있다. `IntegrationService` 는 `enabled=true`인 첫 번째 설정을 Adapter 실행 설정으로 사용한다. 같은 프로토콜의 모든 설정이 `disabled`이면 해당 프로토콜은 실행하지 않는다.

3.7 모니터링/성능관리 API

`MonitoringController` 는 운영자가 현재 처리 현황을 조회할 수 있는 API를 제공한다.

API	설명
<code>GET /api/monitoring/summary</code>	전체 요청 수, 처리 중/완료 요청 수, <code>overallStatus</code> 별 건수, 전체 로그 수, 최근 로그 10건
<code>GET /api/performance/protocols</code>	프로토콜별 전체/성공/실패/타임아웃 건수, 성공률, 평균 실행시간

4. 병렬 처리 구조

4.1 COMPLETABLEFUTURE

프로토콜별 어댑터는 `CompletableFuture.supplyAsync()` 로 실행된다.

예시:

```
futures.put("SFTP", CompletableFuture.supplyAsync(
    () -> sftpAdapterService.execute(requestId, request.getPayload(),
    adapterConfig),
    integrationTaskExecutor));
```

요청에 포함된 프로토콜만 실행 대상이 된다. 단, 인터페이스 설정이 모두 `disabled`인 프로토콜은 `future`에 추가하지 않고 즉시 `DISABLED` 결과로 확정한다.

4.2 INTEGRATIONTASKEXECUTOR

`IntegrationExecutorConfig` 에서 별도 `ExecutorService`를 등록한다.

```
return Executors.newFixedThreadPool(10, threadFactory);
```

스레드 이름은 `integration-adapter-1`, `integration-adapter-2` 형식이다. 장애 분석 시 스레드 덤프에서 통합 어댑터 작업을 구분하기 쉽다.

4.3 35초 TIMEOUT

`CompletableFuture.allOf(...).orTimeout(35, TimeUnit.SECONDS)` 를 사용한다.

동작:

- 모든 future가 35초 안에 끝나면 각 결과를 수집한다.
- 아직 끝나지 않은 future는 `TIMEOUT`, `504`, `35초 내 응답 없음` 결과로 변환한다.
- 이미 끝났지만 예외가 발생한 future는 `FAILED`, `500` 결과로 변환한다.

이 설계는 API 응답과 DB 저장 결과를 `IntegrationService` 에서 한 번만 확정하기 위한 구조다.

5. 트랜잭션 설계

5.1 원칙

`processIntegration()` 전체에는 `@Transactional` 을 붙이지 않는다.

이유:

- SOAP, Kafka, SFTP, Batch, REST 실행은 외부 시스템 호출 또는 블로킹 작업이다.
- 전체 메서드를 트랜잭션으로 묶으면 최대 35초 동안 DB 커넥션을 점유할 수 있다.
- 동시 요청이 늘어나면 커넥션 풀 고갈 위험이 커진다.

5.2 실제 구현

짧은 DB 저장 구간만 `TransactionTemplate` 으로 감싼다.

메서드	트랜잭션 범위
<code>createInitialRequest()</code>	요청 생성, 시작 로그 저장, 상태 PROCESSING 변경
<code>saveFinalResults()</code>	프로토콜 결과 저장, 완료 로그 저장, 상태 COMPLETED 변경
<code>getStatus()</code>	<code>@Transactional(readOnly = true)</code>
<code>getLogs()</code>	<code>@Transactional(readOnly = true)</code>

외부 어댑터 실행은 트랜잭션 밖에서 수행된다.

6. 결과 저장 책임

어댑터는 `ProtocolResultDTO` 만 반환한다.

DB 저장은 하지 않는다.

이 책임을 `IntegrationService` 에 모은 이유:

- timeout 결과와 실제 어댑터 결과가 서로 다른 시점에 저장되는 문제를 막기 위해
- 프로토콜별 최종 결과를 요청당 한 번만 저장하기 위해
- 전체 상태 계산과 로그 저장을 같은 위치에서 관리하기 위해

저장 대상:

테이블	저장 시점
<code>integration_requests</code>	요청 시작, 최종 완료
<code>protocol_results</code>	각 프로토콜 결과 확정 후
<code>system_logs</code>	요청 시작, 프로토콜별 결과, 최종 완료

7. 어댑터 설계

모든 어댑터는 `ProtocolAdapter` 인터페이스를 구현한다.

```
ProtocolResultDTO execute(String requestId, Map<String, Object> payload);

default ProtocolResultDTO execute(
    String requestId,
    Map<String, Object> payload,
    AdapterExecutionConfig config
) {
    return execute(requestId, payload);
}
```

두 번째 메서드는 인터페이스 설정을 `Adapter` 실행에 전달하기 위해 추가한 확장 지점이다. 기존 테스트나 설정 없는 실행 흐름은 2-argument `execute()` 를 그대로 사용할 수 있고, 설정 `row`가 있는 실행은 `AdapterExecutionConfig` 를 전달한다.

`AdapterExecutionConfig` 필드:

필드	의미
<code>protocol</code>	실행 프로토콜
<code>interfaceName</code>	관리 화면에 등록된 인터페이스 이름
<code>endpoint</code>	Adapter별 대상 식별값
<code>timeoutMs</code>	Adapter별 timeout 값

프로토콜별 반영 방식:

프로토콜	endpoint 반영	timeoutMs 반영
SOAP	내부 legacy 호출 대상 식별값으로 로그/결과 메시지에 반영	현재 직접 통신 timeout 없음
Kafka	발행 topic으로 사용	<code>KafkaTemplate.send()</code> 완료 대기 시간
SFTP	업로드 디렉터리로 사용	session/channel connect timeout
Batch	<code>configuredJobName</code> JobParameter로 전달	현재 Job 실행 timeout 없음
REST	내부 legacy 호출 대상 식별값으로 로그/결과 메시지에 반영	현재 직접 통신 timeout 없음

7.1 SOAP

파일:

- `SoapAdapterService.java`
- `LegacySoapService.java`
- `SoapEndpoint.java`
- `integration.xsd`

현재 SOAP 어댑터는 HTTP self-call을 하지 않는다. `LegacySoapService.process()`를 직접 호출한다.

처리:

- payload를 JSON 문자열로 직렬화한다.
- `IntegrationSoapRequest`에 `requestId`, `payload`를 담는다.

3. `LegacySoapService` 에서 처리한다.
4. 성공 시 `SUCCESS` , `200` 결과를 반환한다.

`SoapEndpoint` 와 XSD는 SOAP endpoint 구조를 남겨둔 컴포넌트다. 현재 통합 흐름의 SOAP 어댑터는 직접 legacy service를 호출한다.

`AdapterExecutionConfig.endpoint` 가 전달되면 실제 외부 SOAP URL 호출 대신 현재 내부 legacy 호출 대상 식별값으로 로그와 결과 메시지에 포함한다. 운영 확장 시 이 값을 `WebServiceTemplate` 의 endpoint URI로 사용할 수 있다.

7.2 KAFKA

파일:

- `KafkaAdapterService.java`
- `KafkaConsumerService.java`
- `KafkaConfig.java`

Producer 흐름:

1. payload를 JSON으로 직렬화한다.
2. 메시지를 `requestId|payloadJson` 형식으로 만든다.
3. topic에 key= `requestId` , value= `message` 로 발행한다.
4. 기본 topic은 `integration-events` 다.
5. 인터페이스 설정이 있으면 `AdapterExecutionConfig.endpoint` 를 topic으로 사용한다.
6. 기본 발행 대기 시간은 5초다.
7. 인터페이스 설정이 있으면 `timeoutMs` 를 `future.get(timeoutMs, TimeUnit.MILLISECONDS)` 에 사용한다.
8. 성공 시 `SUCCESS` , 실패 시 `FAILED` 를 반환한다.

Consumer 흐름:

1. `KafkaConsumerService.consume()` 이 `integration-events` 를 구독한다.
2. 메시지에서 `requestId` 를 파싱한다.
3. consumer는 DB에 `ProtocolResult` 를 저장하지 않고 로그만 남긴다.

DLT 처리:

- DLT topic: `integration-events-dlt`
- `DefaultErrorHandler` 가 1초 간격으로 최대 3회 재시도한다.

- 계속 실패하면 `DeadLetterPublishingRecoverer` 가 DLT topic의 partition 0으로 보낸다.
- DLT consumer는 메시지를 받고 `requestId` 파싱 로그만 남긴다.

DLT는 dead letter topic의 약자다. 여러 번 처리에 실패한 메시지를 버리지 않고 따로 보내는 실패 보관함이다.

7.3 SFTP

파일:

- `SftpAdapterService.java`
- `application.yml`
- `docker-compose.yml`
- `docker/sftp/host_keys/*`
- `docker/sftp/init.d/fix-upload-permissions.sh`

처리:

1. payload를 JSON으로 직렬화한다.
2. 파일명을 `finbridge-{requestId}.json` 으로 만든다.
3. JSch session을 생성한다.
4. `StrictHostKeyChecking=yes` 이면 `known_hosts` 를 등록한다.
5. 기본 session connect timeout과 channel connect timeout은 10초다.
6. 인터페이스 설정이 있으면 `timeoutMs` 를 session/channel connect timeout으로 사용한다.
7. 기본 업로드 디렉터리는 `${sftp.upload-dir}` 이다.
8. 인터페이스 설정이 있으면 `endpoint` 를 업로드 디렉터리로 사용한다.
9. finally에서 channel과 session을 disconnect한다.

로컬 Docker SFTP는 host key를 repo에 고정한다. 따라서 컨테이너를 지웠다가 다시 띄워도 `[local-host]:2222` host key가 바뀌지 않는다.

운영 환경에서는 repo의 데모 host key를 사용하면 안 된다. 실제 SFTP 서버의 host key를 `known_hosts` 에 등록해야 한다.

7.4 BATCH

파일:

- `BatchAdapterService.java`
- `BatchJobConfig.java`

- `V3__create_spring_batch_metadata_tables.sql`

처리:

1. `JobParameters` 에 `requestId`, `payload`, `timestamp`, `configuredJobName`, `interfaceName` 을 넣는다.
2. `JobLauncher.run(integrationJob, params)` 를 호출한다.
3. `BatchStatus.COMPLETED` 면 `SUCCESS`, 아니면 `FAILED` 를 반환한다.

`timestamp` `JobParameter`는 같은 Job이 반복 실행될 때 Spring Batch가 같은 `JobInstance`로 판단하지 않게 하기 위한 값이다.

`integrationItemReader()` 는 `@StepScope` 가 붙은 `ListItemReader<String>` 다. Step 실행마다 reader 인스턴스를 새로 만들어 반복 실행 시 첫 실행 이후 데이터가 비는 문제를 피한다.

Spring Batch 메타데이터 테이블은 Flyway V3에서 생성한다.

현재 실제 실행 Job bean은 `integrationJob` 하나다. 인터페이스 설정의 `endpoint`는 동적으로 다른 Job bean을 선택하는 데 사용하지 않고, `configuredJobName` `JobParameter`로 전달한다. 운영 확장 시 `endpoint`를 `JobRegistry` 기반 job name 선택으로 연결할 수 있다.

7.5 REST

파일:

- `RestAdapterService.java`
- `LegacyRestService.java`

현재 REST 어댑터는 `WebClient.block()` 이나 `localhost self-call`을 사용하지 않는다.

`LegacyRestService.echo()` 를 직접 호출한다.

이 구조는 로컬 데모에서 서블릿 스레드가 자기 자신을 다시 호출해 고갈되는 문제를 피하기 위한 선택이다.

`AdapterExecutionConfig.endpoint` 가 전달되면 실제 외부 REST URL 호출 대신 현재 내부 legacy 호출 대상 식별값으로 로그와 결과 메시지에 포함한다. 운영 확장 시 이 값을 `RestClient` 또는 `WebClient` 의 요청 URL로 사용할 수 있다.

8. Kafka 설계

8.1 TOPIC

`KafkaConfig`가 topic을 생성한다.

topic	목적	partition
<code>integration-events</code>	통합 이벤트 발행	3
<code>integration-events-dlt</code>	실패 메시지 보관	1

8.2 DOCKER-COMPOSE LISTENER

Kafka는 Docker 내부 listener와 host listener를 분리한다.

```
KAFKA_ADVERTISED_LISTENERS:
'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
```

Spring Boot 앱은 host에서 `localhost:9092`로 접속한다. Kafka 컨테이너 내부 통신은 `kafka:29092`를 사용한다.

9. SFTP 설계

SFTP는 보안 설정을 로컬 데모에서도 최대한 운영과 비슷하게 가져간다.

구현된 것:

- `StrictHostKeyChecking=yes`
- `${user.home}/.ssh/known_hosts` 사용
- Docker SFTP host key 고정
- session/channel connect timeout 10초
- finally에서 session/channel 정리
- `SFTP_PASSWORD` 등 환경변수 override 지원

known_hosts 최초 등록:

```
mkdir -p "$HOME/.ssh"
chmod 700 "$HOME/.ssh"
ssh-keygen -R "[localhost]:2222" -f "$HOME/.ssh/known_hosts"
ssh-keyscan -T 10 -p 2222 localhost >> "$HOME/.ssh/known_hosts"
chmod 600 "$HOME/.ssh/known_hosts"
```

고정 host key를 사용하므로 일반적인 `docker compose down -v` 이후에도 매번 다시 등록할 필요는 없다. 단, `docker/sftp/host_keys/` 파일을 교체하면 다시 등록해야 한다.

10. Batch 설계

Spring Batch는 `spring.batch.job.enabled=false`로 자동 실행을 끈다.

실행은 `BatchAdapterService`에서 `JobLauncher`로 직접 수행한다.

구성:

- Job: `integrationJob`
- Step: `integrationStep`
- Reader: `@StepScope ListItemReader<String>`
- Processor: `processed-` prefix를 붙이는 단순 처리
- Writer: 처리 항목 로그 기록

메타데이터:

- `BATCH_JOB_INSTANCE`
- `BATCH_JOB_EXECUTION`
- `BATCH_JOB_EXECUTION_PARAMS`
- `BATCH_STEP_EXECUTION`
- `BATCH_*_CONTEXT`
- `BATCH_*_SEQ`

위 테이블은 `V3__create_spring_batch_metadata_tables.sql`에서 생성한다.

11. DB와 Flyway

11.1 도메인 테이블

V1__init.sql:

테이블	목적
integration_requests	통합 요청 단위 저장
protocol_results	프로토콜별 실행 결과 저장
system_logs	요청 생명주기와 이벤트 로그 저장

11.2 인덱스

V2__add_indexes.sql 는 조회 성능을 위한 인덱스를 추가한다.

주요 조회 패턴:

- requestId로 요청 상태 조회
- requestId로 protocol results 조회
- protocol + timestamp로 logs 조회
- timestamp desc로 최근 로그 조회

V4__remove_duplicate_request_id_index.sql 는 request_id UNIQUE 와 중복되는 인덱스를 제거한다.

V5__create_interface_configs.sql 는 인터페이스 등록/설정 테이블을 생성하고 기본 프로토콜 설정을 seed한다.

테이블	목적
interface_configs	프로토콜별 인터페이스 이름, endpoint, enabled, timeoutMs, description 관리

기본 seed 데이터는 SOAP, Kafka, SFTP, Batch, REST 5개 프로토콜의 데모 설정을 포함한다. 이 설정은 /api/interfaces 와 웹 콘솔에서 조회/수정할 수 있으며, IntegrationService 가 실행 전에 조회한다.

11.3 JPA 설정

운영 profile 기본값은 다음 정책을 사용한다.

```
spring:
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
```

스키마 생성은 Flyway가 맡고, Hibernate는 entity와 DB 스키마가 맞는지 검증한다.

12. API 명세

12.1 POST /API/INTEGRATE

요청:

```
{
  "protocols": ["SOAP", "KAFKA", "SFTP", "BATCH", "REST"],
  "payload": {
    "customerId": "demo-customer",
    "amount": 12000,
    "currency": "KRW"
  }
}
```

응답:

```
{
  "requestId": "550e8400-e29b-41d4-a716-446655440000",
  "overallStatus": "ALL_SUCCESS",
  "results": {
    "SOAP": {
      "status": "SUCCESS",
      "responseCode": "200",
      "responseMessage": "SOAP 레거시 처리 완료",
      "executionTimeMs": 12
    },
    "KAFKA": {
      "status": "SUCCESS",
      "responseCode": "200",
      "responseMessage": "Kafka 메시지 발행 완료",
      "executionTimeMs": 41
    }
  },
  "createdAt": "2026-04-26T04:20:10",
  "completedAt": "2026-04-26T04:20:11"
}
```

`createdAt` 은 요청 생성 시각이다.

`completedAt` 은 요청된 모든 프로토콜 결과가 확정되고 최종 상태가 저장된 시각이다.

12.2 GET /API/INTEGRATE/{REQUESTID}

존재하는 요청이면 `IntegrationResponseDTO` 를 반환한다.

없는 요청이면 404를 반환한다.

12.3 GET /API/LOGS

요청:

```
GET /api/logs?limit=10&offset=0
GET /api/logs?protocol=SFTP&limit=10&offset=0
```

응답:

```
{
  "total": 3,
  "limit": 10,
  "offset": 0,
  "logs": [
    {
      "id": 1,
      "requestId": "550e8400-e29b-41d4-a716-446655440000",
      "protocol": "SFTP",
      "eventType": "SUCCESS",
      "eventDetail": "SFTP 처리 결과: SUCCESS",
      "timestamp": "2026-04-26T04:20:11"
    }
  ]
}
```

12.4 POST /API/INTEGRATE/{REQUESTID}/RETRY

body 없이 호출하면 원본 요청에서 `SUCCESS` 가 아닌 프로토콜만 자동으로 재처리한다.

```
POST /api/integrate/550e8400-e29b-41d4-a716-446655440000/retry
```

특정 프로토콜만 재처리할 수도 있다.

```
{
  "protocols": ["SFTP", "KAFKA"]
}
```

응답:

```
{
  "originalRequestId": "550e8400-e29b-41d4-a716-446655440000",
  "retryRequestId": "7abfdb94-f862-43a4-9a79-8abed5a23e10",
  "retryResponse": {
    "requestId": "7abfdb94-f862-43a4-9a79-8abed5a23e10",
    "overallStatus": "ALL_SUCCESS",
    "results": {},
    "createdAt": "2026-04-26T05:10:00",
    "completedAt": "2026-04-26T05:10:01"
  }
}
```


12.5 /API/INTERFACES

등록 요청:

```
{
  "protocol": "SFTP",
  "interfaceName": "Partner SFTP Upload",
  "endpoint": "/upload",
  "enabled": true,
  "timeoutMs": 10000,
  "description": "기관 연계 파일 업로드"
}
```

조회/수정:

```
GET /api/interfaces
GET /api/interfaces?protocol=SFTP
PUT /api/interfaces/{id}
```

`enabled=false`로 변경하면 해당 프로토콜은 통합 요청에서 Adapter 실행 없이 `DISABLED` 결과로 저장된다.

12.6 모니터링과 성능관리

`GET /api/monitoring/summary` 응답 예시:

```
{
  "totalRequests": 20,
  "processingRequests": 0,
  "completedRequests": 20,
  "allSuccess": 15,
  "partialFailure": 4,
  "allFailed": 1,
  "totalLogs": 80,
  "recentLogs": []
}
```

`GET /api/performance/protocols` 응답 예시:

```
[
  {
    "protocol": "SFTP",
    "totalCount": 10,
    "successCount": 9,
    "failedCount": 1,
    "timeoutCount": 0,
    "successRate": 90.0,
    "averageExecutionTimeMs": 120.5
  }
]
```

13. 테스트 전략

13.1 UNIT TEST

주요 단위 테스트:

- IntegrationControllerTest
- IntegrationServiceTest
- InterfaceConfigControllerTest
- InterfaceConfigServiceTest
- KafkaConsumerServiceTest
- MonitoringControllerTest
- MonitoringServiceTest
- IntegrationExecutorConfigTest
- BatchAdapterServiceTest
- KafkaAdapterServiceTest

검증 대상:

- protocol validation
- lowercase normalization
- invalid limit/offset 처리
- overallStatus 계산
- ProtocolResult 중앙 저장
- lifecycle SystemLog 저장
- enabled=false 프로토콜 Adapter 실행 생략

- 인터페이스 설정값 Adapter 전달
- 재처리 대상 프로토콜 선택
- 모니터링/성능 통계 계산
- Kafka 메시지 포맷과 consumer parsing
- Batch JobParameters
- Executor thread name

실행:

```
./gradlew test
```

13.2 H2 기반 SPRING INTEGRATION TEST

`IntegrationApiIT` 는 Spring MVC, Service, Repository, H2 DB 경로를 검증한다.

특징:

- adapter는 mock 처리
- Flyway는 비활성화
- JPA `ddl-auto=create-drop`
- API와 DB 저장 흐름을 빠르게 검증

이 테스트는 외부 시스템 실제 연동 검증이 아니다. 빠른 API/DB 통합 테스트다.

13.3 DOCKER E2E TEST

`RealDockerE2EIT` 는 실제 Docker 서비스를 사용한다.

검증 대상:

- MySQL + Flyway migration
- Kafka publish
- SFTP upload
- Spring Batch job execution
- 5개 프로토콜 전체 `/api/integrate` 성공

실행:

```
RUN_DOCKER_E2E=true ./gradlew test --tests '*RealDockerE2EIT'
```

전제:

- `docker compose up -d`
- SFTP `known_hosts` 등록 완료

14. 주요 장애 대응 포인트

SFTP HOSTKEY HAS BEEN CHANGED

원인:

- `known_hosts`에 저장된 `[localhost]:2222` key와 현재 SFTP 서버 key가 다름

대응:

```
ssh-keygen -R "[localhost]:2222" -f "$HOME/.ssh/known_hosts"
ssh-keyscan -T 10 -p 2222 localhost >> "$HOME/.ssh/known_hosts"
```

현재는 Docker SFTP host key를 repo에 고정했으므로, host key 파일을 바꾸지 않는 한 자주 발생하지 않아야 한다.

SFTP PERMISSION DENIED

원인:

- `/home/finbridge/upload` 소유권 또는 권한 문제

대응:

- `docker/sftp/init.d/fix-upload-permissions.sh` 가 컨테이너 시작 시 권한을 보정한다.
- 문제가 반복되면 `docker compose logs sftp` 와 컨테이너 내부 `/home/finbridge/upload` 권한을 확인한다.

KAFKA HOST APP CONNECTION 실패

원인:

- broker가 host 앱에 `kafka:9092` 를 advertised listener로 알려주는 경우

현재 구성:

- host 앱: `localhost:9092`
- Docker 내부: `kafka:29092`

`docker-compose.yml` 의 `KAFKA_ADVERTISED_LISTENERS` 를 변경할 때 이 분리를 유지해야 한다.

BATCH METADATA TABLE 오류

원인:

- Spring Batch가 필요한 `BATCH_*` 테이블이 없음

현재 구성:

- Flyway V3가 Spring Batch metadata tables를 생성한다.
- `spring.jpa.hibernate.ddl-auto=validate` 이므로 DB 스키마가 없으면 앱 시작 또는 Job 실행 시 실패한다.

/API/LOGS 500 오류

과거 이슈:

- `limit=0` 또는 음수 `offset`이 `OffsetBasedPageRequest` 에서 500을 유발할 수 있었다.

현재 구성:

- Controller에서 `limit <= 0`, `offset < 0` 을 400으로 처리한다.

15. 운영 전 개선 포인트

현재 구현은 로컬 데모와 포트폴리오 검증에 맞춰져 있다. 운영 반영 전에는 아래 항목을 추가로 설계해야 한다.

항목	현재 상태	운영 전 권장
인증/권한	없음	Spring Security, 관리자 권한, 요청자 별 접근 제어
CORS	<code>CORS_ALLOWED_ORIGINS</code> 환경변수 지원	실제 프론트엔드 도메인만 허용
SFTP 비밀번호	환경변수 override 지원, 기본값 있음	Secret Manager 또는 배포 환경변수로 강제
외부 SOAP/REST	내부 legacy service 직접 호출, endpoint는 식별값으로 반영	외부 endpoint 실제 호출, timeout, 인증서, 장애 격리 설정
인터페이스 설정	enabled, endpoint, timeoutMs 실행 반영	인증 방식, headers, retry count, SLA, 담당자 정보 확장
재처리	실패 프로토콜 단위 retry API 구현	재처리 이력 관계 테이블, 횟수 제한, 사유/작업자 기록
알림	미구현	실패/timeout 기준 Slack, Email, SMS 알림
관측성	로그, 요약 모니터링, 프로토콜별 성능 통계	Micrometer, trace, Grafana dashboard, p95/p99 지표
부하 제어	fixed thread pool 10	요청량 기준 queue, rejection policy, rate limit 검토

16. 유지보수 체크리스트

새 프로토콜을 추가할 때:

1. `ProtocolType` enum에 값 추가
2. `IntegrationController.SUPPORTED_PROTOCOLS` 에 값 추가
3. `ProtocolAdapter` 구현체 추가
4. 필요하면 `AdapterExecutionConfig` 의 endpoint/timeout 해석 규칙 정의
5. `IntegrationService.processIntegration()` 에 설정 조회와 future 등록 로직 추가
6. `InterfaceConfig` seed migration 또는 운영 등록 절차 추가
7. `MonitoringService.getProtocolPerformance()` 에서 통계 대상에 포함되는지 확인
8. `ProtocolResult` 저장과 `SystemLog` 저장이 중앙 흐름에 들어오는지 확인
9. unit test, integration test, 필요 시 Docker E2E test 추가

DB 스키마를 바꿀 때:

1. entity 수정
2. Flyway migration 추가
3. `ddl-auto=validate` 기준으로 앱 기동 확인
4. Docker E2E test로 MySQL migration 경로 확인

외부 시스템 설정을 바꿀 때:

1. `application.yml` 기본값 확인
2. test/e2e profile 설정 확인
3. Docker Compose 포트와 advertised listener 확인
4. README와 이 문서의 실행 방법 동기화